

TUGAS PENDAHULUAN MODUL 6
STRUKTUR DATA



Disusun Oleh :

Nurul Maulina Nainggolan

21104053/SE07-01

PROGRAM STUDI REKAYASA PERANGKAT LUNAK

FAKULTAS INFORMATIKA

TELKOM UNIVERAITY PURWOKERTO

2024

MODUL 6

TUGAS PENDAHULUAN MODUL 6 STRUKTUR DATA- GANJIL 2024/2025 “Double Linked List (Bagian Pertama)”

1. Soal 1 : Menambahkan Elemen di Awal dan Akhir DLL

Deskripsi Soal : Buatlah program yang mengizinkan pengguna menambahkan elemen ke dalam Doubly Linked List di awal dan di akhir list..

Instruksi :

- Implementasikan fungsi `insertFirst` untuk menambahkan elemen di awal list.
- Implementasikan fungsi `insertLast` untuk menambahkan elemen di akhir list.
- Tampilkan seluruh elemen dalam list dari depan ke belakang setelah penambahan dilakukan.

Contoh Input :

- Input : Masukkan elemen pertama = 10
- Input : Masukkan elemen kedua di awal = 5
- Input : Masukkan Elemen ketiga di Akhir = 20

Output : DAFTAR ANGGOTA LIST : 5<->10<->20

Pendeklarasian :

```

#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* prev;
    Node* next;
};

Node* head = NULL;

// Fungsi untuk menambahkan elemen di awal
void insertFirst(int data) {
    Node* newNode = new Node();
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = head;

    if (head != NULL) {
        head->prev = newNode;
    }
    head = newNode;
}

// Fungsi untuk menambahkan elemen di akhir
void insertLast(int data) {
    Node* newNode = new Node();
    newNode->data = data;
    newNode->next = NULL;

    if (head == NULL) {
        newNode->prev = NULL;
        head = newNode;
        return;
    }

    Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
    newNode->prev = temp;
}

// Fungsi untuk menampilkan elemen dari depan ke belakang
void displayForward() {
    Node* temp = head;
    cout << "DAFTAR ANGGOTA LIST : ";
    while (temp != NULL) {
        cout << temp->data;
        if (temp->next != NULL) {
            cout << "<->";
        }
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    int firstElement, secondElement, lastElement;

    // Input data sesuai dengan instruksi
    cout << "Masukkan elemen pertama = ";
    cin >> firstElement;
    insertFirst(firstElement);

    cout << "Masukkan elemen kedua = ";
    cin >> secondElement;
    insertFirst(secondElement);

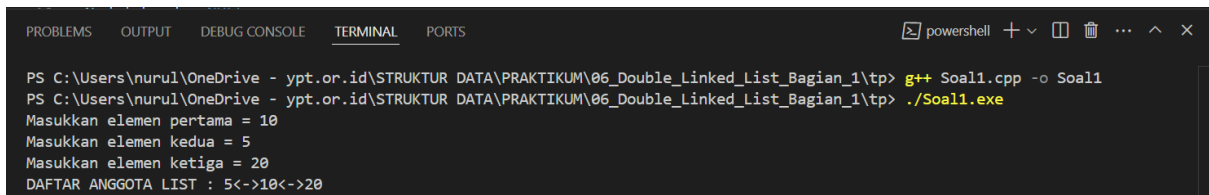
    cout << "Masukkan elemen ketiga = ";
    cin >> lastElement;
    insertLast(lastElement);

    // Tampilkan list dari depan ke belakang
    displayForward();

    return 0;
}

```

Output :



```
PS C:\Users\nurul\OneDrive - ypt.or.id\STRUKTUR DATA\PRAKTIKUM\06_Double_Linked_List_Bagian_1\tp> g++ Soal1.cpp -o Soal1
PS C:\Users\nurul\OneDrive - ypt.or.id\STRUKTUR DATA\PRAKTIKUM\06_Double_Linked_List_Bagian_1\tp> ./Soal1.exe
Masukkan elemen pertama = 10
Masukkan elemen kedua = 5
Masukkan elemen ketiga = 20
DAFTAR ANGGOTA LIST : 5<->10<->20
```

Pada soal ini mengarahkan untuk membuat list berisi 3 yaitu 10, 5, dan 20. Menggunakan pointer prev untuk menunjukkan node sebelumnya dalam list. Selanjutnya menggunakan **insertFirst** untuk menambahkan node baru di awal list dan Jika daftar tidak kosong, node pertama (head) akan memiliki pointer prev yang menunjuk ke node baru. **insertLast** untuk menambahkan node baru pada akhir list dan Jika daftar tidak kosong, fungsi ini akan menelusuri hingga node terakhir dan mengatur pointer next dari node terakhir untuk menunjuk ke node baru, serta prev dari node baru untuk menunjuk ke node terakhir. **DisplayForward** untuk menampilkan semua elemen dalam daftar dari depan ke belakang. Program ini menunjukkan cara dasar untuk menggunakan DLL, termasuk cara menambah elemen di awal dan akhir, serta cara menampilkan elemen-elemen yang ada di dalamnya.

2. Soal 2 : Menghapus Elemen di Awal dan Akhir DLL

Deskripsi Soal : Buatlah program yang memungkinkan pengguna untuk menghapus elemen terakhir dalam Doubly Linked List.

Instruksi :

- Implementasikan fungsi `'deleteFirst'` untuk menghapus elemen pertama.
- Implementasikan fungsi `'deleteLast'` untuk menghapus elemen terakhir.
- Tampilkan seluruh elemen dalam list setelah penghapusan dilakukan.

Contoh Input :

- Input: Masukkan elemen pertama = 10
- Input: Masukkan elemen kedua di akhir = 15
- Input: Masukkan elemen ketiga di akhir = 20
- Hapus elemen pertama dan terakhir.

Ouput : DAFTAR ANGGOTA LIST SETELAH PENGHAPUSAN : 15

Pendeklarasian :

```

#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* prev;
    Node* next;
};

Node* head = NULL;

// Fungsi untuk menambahkan elemen di awal
void insertFirst(int data) {
    Node* newNode = new Node();
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = head;

    if (head != NULL) {
        head->prev = newNode;
    }
    head = newNode;
}

// Fungsi untuk menambahkan elemen di akhir
void insertLast(int data) {
    Node* newNode = new Node();
    newNode->data = data;
    newNode->next = NULL;

    if (head == NULL) {
        newNode->prev = NULL;
        head = newNode;
        return;
    }

    Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
    newNode->prev = temp;
}

// Fungsi untuk menghapus elemen pertama
void deleteFirst() {
    if (head == NULL) return;

    Node* temp = head;
    head = head->next;
    if (head != NULL) {
        head->prev = NULL;
    }
    delete temp;
}

// Fungsi untuk menghapus elemen terakhir
void deleteLast() {
    if (head == NULL) return;

    Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    if (temp->prev != NULL) {
        temp->prev->next = NULL;
    } else {
        head = NULL;
    }
    delete temp;
}

// Fungsi untuk menampilkan elemen dari depan ke belakang
void displayForward() {
    Node* temp = head;
    cout << "DAFTAR ANGGOTA LIST SETELAH PENGHAPUSAN : ";
    while (temp != NULL) {
        cout << temp->data;
        if (temp->next != NULL) {
            cout << "<->";
        }
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    int firstElement, secondElement, lastElement;

    // Input data sesuai dengan instruksi
    cout << "Masukkan elemen pertama = ";
    cin >> firstElement;
    insertFirst(firstElement);

    cout << "Masukkan elemen kedua di akhir = ";
    cin >> secondElement;
    insertLast(secondElement);

    cout << "Masukkan elemen ketiga di akhir = ";
    cin >> lastElement;
    insertLast(lastElement);

    // Menghapus elemen pertama dan terakhir
    deleteFirst();
    deleteLast();

    // Tampilkan list dari depan ke belakang
    displayForward();

    return 0;
}

```

Output

```
PS C:\Users\nurul\OneDrive - ypt.or.id\STRUKTUR DATA\PRAKTIKUM\06_Double_Linked_List_Bagian_1> cd tp
PS C:\Users\nurul\OneDrive - ypt.or.id\STRUKTUR DATA\PRAKTIKUM\06_Double_Linked_List_Bagian_1\tp> g++ Soal2.cpp -o Soal2
PS C:\Users\nurul\OneDrive - ypt.or.id\STRUKTUR DATA\PRAKTIKUM\06_Double_Linked_List_Bagian_1\tp> ./Soal2.exe
Masukkan elemen pertama = 10
Masukkan elemen kedua di akhir = 20
Masukkan elemen ketiga di akhir = 30
DAFTAR ANGGOTA LIST SETELAH PENGHAPUSAN : 20
PS C:\Users\nurul\OneDrive - ypt.or.id\STRUKTUR DATA\PRAKTIKUM\06_Double_Linked_List_Bagian_1\tp> █
```

insertFirst untuk menambahkan node baru di awal list dan Jika daftar tidak kosong, node pertama (head) akan memiliki pointer prev yang menunjuk ke node baru. **insertLast** untuk menambahkan node baru pada akhir list dan Jika daftar tidak kosong, fungsi ini akan menelusuri hingga node terakhir dan mengatur pointer next dari node terakhir untuk menunjuk ke node baru, serta prev dari node baru untuk menunjuk ke node terakhir. **DisplayForward** untuk menampilkan semua elemen dalam daftar dari depan ke belakang. **deleteFirst** menghapus node pertama dari daftar, jika daftar kosong tidak melakukan apa-apa, Jika ada node, fungsi menyimpan node pertama dalam variabel temp, kemudian memperbarui head untuk menunjuk ke node berikutnya. Jika node berikutnya ada, maka pointer prev dari node tersebut diatur menjadi NULL. **deleteLast** untuk menghapus node terakhir dari daftar, Fungsi ini menelusuri hingga node terakhir, dan jika node terakhir memiliki node sebelumnya, pointer next dari node sebelumnya diatur menjadi NULL. Jika node terakhir adalah satu-satunya node, maka head diatur menjadi NULL.

3. Soal 3 : Menampilkan Elemen dari Depan ke Belakang dan Sebaliknya

Deskripsi soal : Buatlah program yang memungkinkan pengguna memasukkan Beberapa elemen ke dalam Doubly Linked List. Setelah elemen dimasukkan, tampilkan seluruh elemen dalam list dari depan ke belakang, kemudian dari belakang ke depan.

Instruksi :

- Implementasikan fungsi untuk menampilkan elemen dari depan ke belakang.
- Implementasikan fungsi untuk menampilkan elemen dari belakang ke depan.
- Tambahkan 4 elemen ke dalam list dan tampilkan elemen tersebut dalam dua arah.

Contoh Input :

- Input : Masukkan 4 elemen secara berurutan : 1, 2, 3, 4

Output :

- Daftar elemen dari depan ke belakang : 1 <-> 2 <-> 3 <-> 4
- Daftar elemen dari belakang ke depan : 4 <-> 3 <-> 2 <-> 1

Pendeklarasian :

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* prev;
    Node* next;
};

Node* head = NULL;

// Fungsi untuk menambahkan elemen di akhir
void insertLast(int data) {
    Node* newNode = new Node();
    newNode->data = data;
    newNode->next = NULL;

    if (head == NULL) {
        newNode->prev = NULL;
        head = newNode;
        return;
    }

    Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
    newNode->prev = temp;
}

// Fungsi untuk menampilkan elemen dari depan ke belakang
void displayForward() {
    Node* temp = head;
    cout << "Daftar elemen dari depan ke belakang : ";
    while (temp != NULL) {
        cout << temp->data;
        if (temp->next != NULL) {
            cout << " <-> ";
        }
        temp = temp->next;
    }
    cout << endl;
}

// Fungsi untuk menampilkan elemen dari belakang ke depan
void displayBackward() {
    Node* temp = head;
    if (temp == NULL) return;

    // Pindah ke node terakhir
    while (temp->next != NULL) {
        temp = temp->next;
    }

    cout << "Daftar elemen dari belakang ke depan : ";
    while (temp != NULL) {
        cout << temp->data;
        if (temp->prev != NULL) {
            cout << " <-> ";
        }
        temp = temp->prev;
    }
    cout << endl;
}

int main() {
    int n, data;

    cout << "Masukkan 4 elemen secara berurutan: ";
    for (int i = 0; i < 4; i++) {
        cin >> data;
        insertLast(data);
    }

    // Tampilkan list dari depan ke belakang
    displayForward();

    // Tampilkan list dari belakang ke depan
    displayBackward();

    return 0;
}
```

Output :

```
PS C:\Users\nurul\OneDrive - ypt.or.id\STRUKTUR DATA\PRAKTIKUM\06_Double_Linked_List_Bagian_1> cd tp
PS C:\Users\nurul\OneDrive - ypt.or.id\STRUKTUR DATA\PRAKTIKUM\06_Double_Linked_List_Bagian_1\tp> g++ Soal3.cpp -o Soal3
PS C:\Users\nurul\OneDrive - ypt.or.id\STRUKTUR DATA\PRAKTIKUM\06_Double_Linked_List_Bagian_1\tp> ./Soal3.exe
Masukkan 4 elemen secara berurutan: 1 2 3 4
Daftar elemen dari depan ke belakang : 1 <-> 2 <-> 3 <-> 4
Daftar elemen dari belakang ke depan : 4 <-> 3 <-> 2 <-> 1
PS C:\Users\nurul\OneDrive - ypt.or.id\STRUKTUR DATA\PRAKTIKUM\06_Double_Linked_List_Bagian_1\tp> █
```

insertLast untuk menambahkan elemen baru di akhir linked list. Sebuah node baru dibuat dan diinisialisasi dengan data yang diberikan. Pointer next dari node baru diatur menjadi NULL. Jika linked list kosong (head adalah NULL), node baru menjadi head. Jika linked list tidak kosong, fungsi menelusuri linked list hingga menemukan node terakhir (yang next-nya adalah NULL). Setelah menemukan node terakhir, pointer next dari node terakhir diatur untuk menunjuk ke node baru, dan pointer prev dari node baru diatur untuk menunjuk ke node terakhir. Fungsi **displayForward** Fungsi ini digunakan untuk menampilkan elemen-elemen dalam linked list dari depan ke belakang. Fungsi mulai dari head dan mencetak nilai setiap node. Jika ada node berikutnya, ditambahkan tanda " <-> " di antara elemen untuk menunjukkan hubungan antar node. Fungsi **displayBackward** Fungsi ini digunakan untuk menampilkan elemen-elemen dalam linked list dari belakang ke depan. Pertama, fungsi memeriksa apakah linked list kosong. Jika kosong, fungsi tidak melakukan apa-apa dan keluar. Jika ada elemen, fungsi akan menelusuri linked list hingga mencapai node terakhir. Setelah mencapai node terakhir, fungsi mencetak nilai dari node terakhir ke node pertama, menggunakan pointer prev untuk bergerak mundur.